

Sistema Híbrido de Recuperación de Información para Literatura Científica

John Mauris López Ramos

Universidad de La Habana

johnmauris04@gmail.com •  @John3_141592

Abstract. Este documento presenta el diseño e implementación de un sistema de recuperación de información orientado al dominio de la investigación científica. El sistema integra adquisición de datos desde arXiv, indexación léxica y semántica, expansión de consultas, ranking híbrido, generación aumentada por recuperación (RAG), sistema de recomendación y una interfaz visual para la interacción con el usuario.

Keywords: Recuperación de información · BM25F · Embeddings · RAG · arXiv

1 Introducción

La creciente producción de literatura científica genera grandes volúmenes de información con alta densidad terminológica, lo que dificulta su recuperación mediante métodos manuales. En este contexto, los SRI enfrentan el reto de consultas breves y ambiguas frente a documentos extensos, así como una brecha semántica que los enfoques puramente léxicos no logran resolver, lo que motiva el uso de estrategias híbridas que combinen representaciones léxicas y semánticas para mejorar la calidad del ranking.

Este trabajo presenta un sistema de recuperación híbrido para literatura científica que integra indexación vectorial, expansión de consultas y generación aumentada por recuperación (RAG). El sistema incorpora además adquisición dinámica de documentos y un módulo de búsqueda web para ampliar el corpus cuando es necesario, junto con una interfaz visual para la interacción del usuario y un motor de recomendaciones basado en historial. El objetivo es proporcionar una herramienta integral que no solo recupere documentos relevantes, sino que también genere respuestas contextualizadas mediante modelos de lenguaje, facilitando la exploración del conocimiento científico.

2 Dominio de Aplicación

El dominio de la investigación científica se caracteriza por documentos extensos, densos en información y con alta variabilidad terminológica, donde la relevancia no depende únicamente de coincidencias léxicas. Además, las consultas suelen ser breves o ambiguas, generando un desajuste entre la intención del usuario y

la representación textual de los documentos. Estas particularidades requieren un enfoque híbrido de recuperación que combine métodos léxicos para capturar coincidencias exactas, recuperación densa para similitud semántica, expansión de consultas para reducir ambigüedad y técnicas de *reranking* para integrar múltiples señales y obtener un orden de resultados más robusto. El corpus utilizado proviene de documentos científicos en formato PDF de arXiv, una fuente adecuada por su cobertura multidisciplinaria, actualización constante y capacidad de escalabilidad.

3 Corpus y Adquisición de Datos

3.1 Crawling

El proceso de *crawling* se encarga de la exploración sistemática de recursos web con el objetivo de localizar documentos y enlaces relevantes dentro del dominio de interés. En este sistema, la exploración se realiza respetando estrictamente las políticas definidas en el archivo `robots.txt` de arXiv, considerando tanto las rutas permitidas (*allowed URLs*) como las restricciones de frecuencia de acceso (*crawl_delay*), con el fin de garantizar un comportamiento ético y conforme a las normas del servidor.

A partir de la página inicial del sitio, el sistema identifica y extrae URLs candidatas, filtrando posteriormente aquellas que cumplen con las políticas de acceso y que corresponden a contenido reciente, típicamente asociado a rutas con el patrón `/recent`.

3.2 Scraping

El proceso de *scraping* opera sobre las páginas previamente obtenidas mediante el *crawling*, analizando su contenido para identificar enlaces directos a documentos en formato PDF, típicamente asociados a rutas con el patrón `/pdf`. Durante esta etapa se respeta el *crawl_delay* definido por el servidor, asegurando un acceso controlado a los recursos.

El módulo se encarga de gestionar el almacenamiento de los enlaces identificados, así como de la descarga efectiva de los documentos hasta un límite predefinido de 2000 PDFs. Adicionalmente, se registran las URLs correspondientes a los documentos descargados.

4 Estadísticas del Corpus

El corpus construido está compuesto por un total de 2000 documentos científicos en formato PDF, con un tamaño promedio de 3.02 MB por archivo. En conjunto, el corpus contiene aproximadamente 9,750,076 tokens "útiles", con un promedio de 4875.04 tokens por documento, lo que evidencia la alta densidad informativa característica del dominio científico.

A nivel de segmentación, los documentos fueron divididos en un total de 22,897 *chunks*, con un promedio de 11.45 fragmentos por documento. Esta granularidad permite mejorar la precisión en la recuperación, facilitando la localización de información relevante dentro de documentos extensos. Asimismo, el vocabulario total del corpus asciende a 362,524 términos únicos, reflejando la diversidad terminológica y la riqueza léxica propia de la literatura científica.

La Figura 1 muestra la distribución de los documentos según su extensión, permitiendo observar la variabilidad en la longitud de los textos procesados.

Por otra parte, la Figura 2 presenta una representación visual de los términos más relevantes del corpus, donde el tamaño de cada palabra está ponderado por su importancia relativa calculada mediante la métrica TF-IDF.

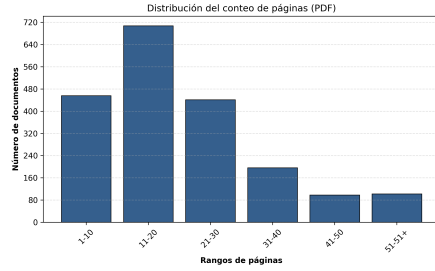


Fig. 1. Distribución de documentos según su extensión.

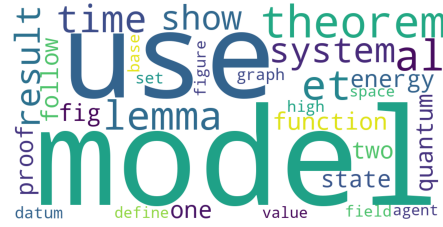


Fig. 2. Nube de palabras basada en puntuaciones TF-IDF.

En conjunto, estas estadísticas evidencian que el corpus presenta una escala y complejidad adecuadas para evaluar sistemas de recuperación de información en el dominio científico, tanto desde una perspectiva léxica como semántica.

5 Preprocesamiento del Texto

El procesamiento de documentos científicos en formato PDF requiere una fase de normalización lingüística que permita transformar el texto bruto en una representación adecuada para su posterior indexación y recuperación. Dada la naturaleza de este dominio, donde abundan estructuras complejas, referencias bibliográficas y variabilidad terminológica, resulta necesario reducir el ruido y homogenizar el contenido textual. Esta etapa garantiza que tanto los modelos léxicos como los semánticos operen sobre datos consistentes, mejorando la calidad de las representaciones y, en consecuencia, la efectividad del sistema de recuperación.

El preprocesamiento se estructura como una secuencia de etapas claramente definidas:

- Conversión del texto a codificación UTF-8 para eliminar caracteres inválidos.

- Normalización a minúsculas.
- Eliminación de citas mediante expresiones regulares.
- Tokenización del texto utilizando *spaCy*, extrayendo lemas alfabéticos.
- Eliminación de *stop words* empleando recursos de *NLTK*.
- Aplicación de *stemming* mediante el algoritmo *Snowball*.
- Segmentación de los tokens en fragmentos (*chunks*) de tamaño fijo con solapamiento.

6 Modelo de Recuperación

6.1 Recuperación Léxica (BM25F)

La recuperación léxica se basa en la coincidencia de términos entre la consulta y los documentos, utilizando el modelo BM25F como función de ranking. Este enfoque resulta especialmente efectivo para capturar precisión terminológica y relevancia basada en la frecuencia y distribución de palabras clave dentro de los documentos. Sin embargo, su principal limitación radica en que no modela relaciones semánticas, por lo que puede fallar ante variaciones lingüísticas o consultas conceptuales.

Modelo BM25F BM25 es un modelo probabilístico de recuperación de información que estima la relevancia de un documento en función de la frecuencia de los términos de la consulta, ajustada por la longitud del documento y la rareza global de los términos en el corpus. BM25F extiende este modelo incorporando la estructura interna del documento mediante *fields* (campos), permitiendo ponderar de forma diferenciada secciones como título, resumen y contenido.

En BM25F, la frecuencia efectiva de un término se define como una combinación ponderada por campo:

$$f(t, D) = \sum_{c \in C} w_c \cdot \frac{f(t, D_c)}{1 - b_c + b_c \cdot \frac{|D_c|}{avgdl_c}}$$

donde t representa el término de la consulta, D el documento completo y C el conjunto de campos del documento (por ejemplo, *title*, *abstract* y *content*). Para cada campo c , $f(t, D_c)$ denota la frecuencia del término t dentro de ese campo específico. El parámetro w_c es el peso asignado al campo c , que controla su importancia relativa en el cálculo final. El término $|D_c|$ representa la longitud del campo c , mientras que $avgdl_c$ es la longitud promedio de ese mismo campo en el corpus. El parámetro b_c controla el grado de normalización por longitud dentro de cada campo, ajustando el impacto de documentos o secciones más extensas.

6.2 Recuperación Semántica (Dense Embeddings)

La recuperación semántica se basa en representaciones vectoriales densas generadas mediante redes neuronales profundas, utilizando *Sentence Transformers*

[3], en particular el modelo *multi-qa-mpnet-base-dot-v1*. Este modelo, basado en arquitecturas Transformer, codifica tanto consultas como *chunks* de texto en vectores de dimensión fija dentro de un espacio semántico continuo, donde la proximidad entre vectores refleja similitud conceptual, permitiendo recuperar información relevante incluso sin coincidencias léxicas explícitas, aunque con posible pérdida de precisión en términos altamente técnicos o específicos.

Para la indexación, cada *chunk* es transformado en un embedding y almacenado en un índice vectorial FAISS [9]. Los vectores son normalizados para permitir el uso del producto interno como equivalente a la similitud coseno. En tiempo de consulta, la entrada del usuario se codifica en el mismo espacio vectorial y se realiza una búsqueda de vecinos más cercanos en el índice, retornando los fragmentos con mayor similitud según un umbral definido, lo que permite una recuperación eficiente en espacios de alta dimensionalidad.

Modelo de Similitud Vectorial El modelo de recuperación semántica se basa en la comparación entre vectores normalizados en un espacio euclidiano. Dado un embedding de consulta $q \in \mathbb{R}^d$ y un embedding de documento $d_i \in \mathbb{R}^d$, ambos normalizados, la similitud se calcula mediante el producto interno:

$$\text{sim}(q, d_i) = q \cdot d_i = \sum_{j=1}^d q_j \cdot d_{i,j}$$

Dado que los vectores están normalizados a norma unitaria, esta operación es equivalente a la similitud del coseno:

$$\text{sim}(q, d_i) = \cos(\theta) = \frac{q \cdot d_i}{\|q\| \|d_i\|}$$

El proceso de recuperación consiste en resolver el problema de *nearest neighbor search*, es decir, encontrar los k vectores d_i que maximizan $\text{sim}(q, d_i)$. Para ello, se utiliza un índice FAISS con métrica de producto interno, optimizado para búsquedas eficientes en grandes colecciones de vectores. Finalmente, se aplica un umbral de similitud para filtrar resultados poco relevantes y se ordenan los candidatos en función de su puntuación.

6.3 Recuperación Híbrida

La recuperación híbrida combina los enfoques léxico y semántico para aprovechar sus fortalezas complementarias, lo cual resulta especialmente necesario en el dominio científico. El sistema genera dos listas de resultados independientes, una por cada modelo, que posteriormente se integran mediante un mecanismo de fusión de rankings basado en RRF. Esta estrategia permite obtener un orden final más robusto y consistente, mejorando la calidad global de la recuperación.

7 Persistencia

El sistema emplea múltiples mecanismos de persistencia especializados según el tipo de dato. Los embeddings densos se almacenan en un índice FAISS, donde se guardan como vectores normalizados para permitir búsquedas eficientes por similitud semántica. En paralelo, Whoosh gestiona el índice léxico, almacenando los documentos estructurados en campos (título, resumen, contenido) para su recuperación mediante BM25F.

Adicionalmente, los fragmentos de texto (*chunks*) se almacenan en una base de datos SQLite, lo que permite su recuperación posterior durante el proceso de RAG. Finalmente, los metadatos asociados a los embeddings (como identificadores de chunks y documentos) se serializan mediante *pickle*, facilitando la correspondencia entre los vectores recuperados y su contenido original.

8 Ranking y Posicionamiento

El módulo de ranking y posicionamiento recibe como entrada dos conjuntos de candidatos generados por los subsistemas de recuperación: el modelo léxico basado en BM25F y el modelo semántico basado en embeddings densos. Ambos recuperadores no devuelven todo el corpus indexado, sino subconjuntos filtrados de documentos potencialmente relevantes que serán posteriormente fusionados.

En el caso de BM25F, cada documento recibe un score de relevancia obtenido mediante el motor Whoosh. A partir del mejor score de la consulta (*best score*) se define un umbral dinámico $score_threshold = best \cdot 0.9$, de modo que solo se conservan los documentos cuyo score sea al menos el 90% del máximo, reduciendo así el conjunto a los más relevantes en términos léxicos.

Para el recuperador semántico, la similitud entre la consulta y los *chunks* se calcula en el espacio vectorial de embeddings, obteniendo un score de similitud. En este caso se aplica un umbral fijo $chunk_similarity_threshold = 0.5$, conservando únicamente los fragmentos cuya similitud supera dicho valor, lo que garantiza un nivel mínimo de coherencia semántica.

De este modo, ambos subsistemas entregan conjuntos filtrados de alta relevancia que sirven como entrada al proceso de fusión y ranking final.

8.1 Ranking algorítmico (RRF)

El sistema utiliza *Reciprocal Rank Fusion* (RRF) como estrategia de fusión de rankings para integrar los resultados provenientes de los recuperadores léxico (BM25F) y semántico. Este método no requiere que los rankings compartan el mismo conjunto de documentos y prioriza aquellos que aparecen en posiciones altas en múltiples listas, generando una ordenación final más robusta y menos dependiente de un único modelo de recuperación.

RRF [4] asigna a cada documento una puntuación en función de su posición en cada ranking. Dado un conjunto de rankings $R = \{r_1, r_2, \dots, r_m\}$, la puntuación final de un documento d se define como:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

donde $\text{rank}_r(d)$ representa la posición del documento d en el ranking r , y k es una constante positiva (usualmente $k = 60$) que controla la caída de contribución de posiciones inferiores. Este mecanismo favorece la consistencia entre rankings, reforzando documentos recurrentemente bien posicionados sin penalizar su ausencia en alguna de las listas.

8.2 Posicionamiento visual

Los resultados recuperados se presentan al usuario en orden descendente de relevancia, priorizando aquellos documentos con mayor puntuación final. Adicionalmente, el sistema permite no solo la descarga de los documentos PDF recuperados, sino también una previsualización (*preview*) de su contenido, lo que facilita una inspección rápida sin necesidad de abandonar el entorno de consulta. Esta estructura mejora la exploración interactiva del corpus y permite al usuario evaluar la pertinencia de los resultados de forma más eficiente.

9 Expansión de Consultas

El sistema implementa un mecanismo de expansión de consultas basado en retroalimentación implícita de la primera fase de recuperación (formalmente desarrollado en [5]), cuyo comportamiento conceptual se ilustra en la Figura 3. Inicialmente, se realiza una búsqueda con la consulta original, a partir de la cual se obtienen *chunks* candidatos que presentan coincidencia o alta similitud semántica. Posteriormente, el contenido de estos fragmentos se combina con la consulta inicial para construir una representación enriquecida, la cual es nuevamente vectorizada mediante el modelo de embeddings, generando una nueva representación más informada del contexto de búsqueda.

10 Módulo RAG

El sistema integra un esquema de *Retrieval-Augmented Generation* (RAG [6]) con el objetivo de no limitarse únicamente a la recuperación de documentos, sino también generar respuestas enriquecidas a partir de la evidencia obtenida. En este enfoque, la consulta del usuario es primero procesada y expandida para mejorar su representatividad semántica. Posteriormente, se construye un contexto a partir de los *chunks* relevantes recuperados por los módulos de búsqueda, los cuales son agregados en una representación textual estructurada.

Este contexto es posteriormente incorporado a un *prompt* que se envía a un modelo generativo alojado en Cloudflare [10], `@cf/meta/llama-3.1-8b-instruct`. El modelo produce una respuesta condicionada por la evidencia recuperada, diferenciando explícitamente entre información derivada del contexto y conocimiento

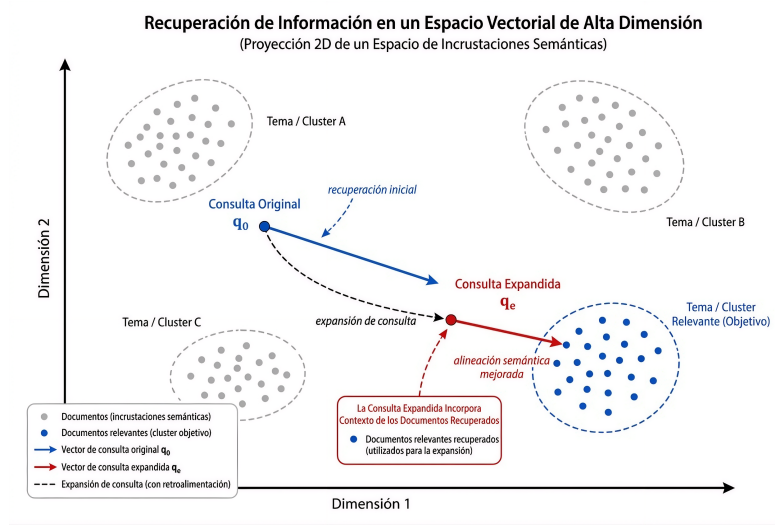


Fig. 3. Efecto de la expansión de consultas en el espacio vectorial semántico. La consulta original se reubica mediante la incorporación de contexto recuperado, mejorando su alineación con documentos relevantes.

general del modelo cuando sea necesario. De este modo, la respuesta no se genera únicamente a partir de memoria paramétrica, sino que está fundamentada en documentos recuperados del corpus.

11 Módulo de Búsqueda Web

El sistema integra un módulo de búsqueda web que permite la ampliación dinámica del corpus cuando los resultados locales no alcanzan un umbral mínimo de relevancia o cantidad. En estos casos, se activa una búsqueda externa mediante la API de arXiv basada en la consulta del usuario, a partir de la cual se descargan nuevos documentos en PDF que se incorporan al flujo de procesamiento e indexación. Este proceso respeta las políticas de `robots.txt`, incluyendo rutas permitidas y restricciones de acceso, garantizando un comportamiento conforme a las normas del servidor. En el dominio científico, donde la literatura evoluciona rápidamente y la cobertura local puede ser incompleta, este mecanismo actúa como un sistema de actualización continua que mejora la pertinencia y cobertura de las respuestas.

12 Sistema de Recomendación

El sistema incorpora un módulo de recomendación (tomando como referencia [7]) basado en el historial de interacciones del usuario, a partir del cual se construye un perfil implícito de intereses. Con este perfil, se sugieren documentos

relacionados con consultas y resultados previos, priorizando similitud temática. Estas recomendaciones funcionan como una capa adicional sobre el sistema de recuperación, aportando personalización sin afectar el ranking principal, lo cual es especialmente útil en búsquedas científicas iterativas y exploratorias.

13 Interfaz Visual

La interfaz visual del sistema fue desarrollada con Streamlit y se organiza en tres vistas principales dedicadas a recomendación, recuperación de información e interacción con el módulo RAG, accesibles mediante una barra de navegación común. Esta separación permite desacoplar las funcionalidades principales del sistema, reduciendo la carga cognitiva del usuario y favoreciendo un flujo de interacción más claro. La vista de recomendaciones presenta documentos sugeridos a partir del historial de uso; la de recuperación permite realizar búsquedas, visualizar resultados y descargar documentos; y la vista tipo chat ofrece una experiencia conversacional para explorar el contenido generado por el sistema RAG. El diseño adopta una estética minimalista basada en tonos neutros con acentos morados para elementos interactivos, priorizando la visibilidad de la búsqueda, los resultados y las respuestas generadas mediante una jerarquía visual que mejora la legibilidad y la exploración de la información.

14 Documentación Técnica Completa del Sistema

El sistema se implementa bajo una arquitectura modular por capas, donde cada componente del flujo de recuperación de información cumple una función específica. La orquestación general es gestionada por un *backend controller*, encargado de coordinar las operaciones del sistema, mientras que el *frontend* en Streamlit se limita a la interacción con el usuario.

El flujo comienza en la interfaz, donde el usuario envía una consulta o interacción. Esta es procesada por el *backend controller*, que determina la operación correspondiente: búsqueda, recomendación o generación RAG. La entrada puede ser expandida y luego enviada al módulo de ranking, el cual combina recuperación léxica (BM25F) y semántica (embeddings densos). Los resultados de ambos enfoques se fusionan mediante RRF. Si la evidencia recuperada no supera un umbral de relevancia, se activa automáticamente la búsqueda web, que descarga nuevos documentos, los procesa e indexa nuevamente para reintegrarlos al sistema de recuperación. Finalmente, los resultados se devuelven al frontend para su visualización.

De forma independiente, el sistema incluye procesos de construcción del corpus basados en *crawling* y *scraping*, junto con la integración de la API de arXiv, los cuales generan la base documental inicial fuera del flujo interactivo. Sobre estos documentos, el módulo de indexación construye y actualiza los índices BM25F y vectoriales, asegurando su disponibilidad para los procesos de recuperación.

14.1 Arquitectura por Capas

- **Capa de Adquisición:** integra *crawling*, *scraping* y la API de arXiv para la recolección de documentos y construcción del corpus inicial.
- **Capa de Procesamiento:** realiza normalización, limpieza, tokenización, eliminación de ruido, generación de hashes y segmentación en *chunks*.
- **Capa de Indexación:** construye y mantiene los índices léxicos (BM25F) y vectoriales (embeddings), además de persistir *chunks* y representaciones intermedias.
- **Capa de Recuperación:** integra BM25F y recuperación densa, expansión de consultas y fusión de resultados mediante RRF.
- **Capa de Generación y Presentación:** combina el módulo RAG con la interfaz Streamlit para visualización de resultados, respuestas generadas y navegación del usuario.
- **Capa de Personalización:** utiliza el historial de interacción para alimentar el sistema de recomendación y adaptar los resultados a los intereses del usuario.

15 Manual de Usuario

Este sistema está diseñado para ser ejecutado como una aplicación web local, sin requerir conocimiento del código fuente.

15.1 Construcción y Actualización del Corpus

La construcción del corpus desde cero se realiza mediante el módulo de adquisición de datos. Para activarlo, es necesario descomentar la función `create_corpus()` en el archivo `arxiv_corpus.py` y ejecutar: `python3 -m data_ingestion.arxiv_corpus`

Este proceso descarga documentos desde arXiv respetando restricciones como *crawl_delay*, límites de descarga y ancho de banda, por lo que puede tardar varias horas.

Una vez descargado el corpus, los índices deben ser generados o actualizados. Para ello se debe descomentar la función `build_indexes()` en `build_indexes.py` y ejecutar: `python3 -m indexing.build_indexes`

Este paso construye tanto el índice léxico como el índice vectorial necesarios para la recuperación de información.

15.2 Uso del Sistema

El usuario interactúa con el sistema exclusivamente a través de tres vistas independientes accesibles desde la barra de navegación superior. En cada una de ellas, la interacción se reduce a acciones simples: navegación entre páginas, escritura en una barra de búsqueda o interacción mediante un chat, sin necesidad de configurar parámetros adicionales.

En la vista de recuperación de información, el usuario introduce una consulta en la barra de búsqueda. A partir de esta entrada, el sistema ejecuta el

proceso de recuperación y presenta una lista de documentos relevantes. Cada resultado puede ser previsualizado o descargado en formato PDF, permitiendo una exploración directa del contenido.

En la vista de recomendaciones, el usuario no realiza acciones adicionales más allá de la navegación, ya que los documentos sugeridos se generan automáticamente a partir del historial de interacción. Finalmente, en la vista de chat RAG, el usuario escribe preguntas en un entorno conversacional. El sistema responde con una salida generada a partir de información recuperada del corpus, presentada como una respuesta textual continua, independiente del listado de documentos.

16 Discusión

16.1 Bondades del Sistema

El sistema propuesto integra de forma coherente múltiples enfoques de recuperación de información, combinando modelos léxicos basados en BM25F con representaciones semánticas mediante embeddings densos. Esta hibridación permite capturar tanto coincidencias terminológicas exactas como similitud conceptual, lo que resulta especialmente adecuado para el dominio de la literatura científica.

Adicionalmente, el uso de un corpus real obtenido desde arXiv aporta relevancia práctica al sistema, mientras que la incorporación de un mecanismo de búsqueda web dinámica permite ampliar la cobertura de información cuando el repositorio local no es suficiente. La inclusión de un módulo RAG añade capacidad de síntesis generativa basada en evidencia recuperada, mejorando la expresividad del sistema sin perder trazabilidad.

16.2 Limitaciones y Soluciones

A pesar de los resultados obtenidos, el sistema presenta algunas limitaciones que constituyen oportunidades de mejora. Su rendimiento depende en gran medida de la calidad y cobertura del corpus, por lo que resulta conveniente incorporar nuevas fuentes de información. Asimismo, el ruido introducido durante la extracción de texto de documentos PDF puede degradar la calidad de los índices construidos, lo que podría mitigarse mediante técnicas más robustas de parsing, limpieza y segmentación. La expansión de consultas, aunque mejora el recall, puede introducir resultados irrelevantes cuando la recuperación inicial es deficiente; por ello, sería beneficioso incorporar mecanismos más avanzados de desambiguación semántica y contextualización. Finalmente, las respuestas generadas por el módulo RAG pueden incluir inferencias no completamente respaldadas por la evidencia recuperada, por lo que futuras mejoras deberían centrarse en reforzar el diseño del prompt y los mecanismos de control del contexto para aumentar la fidelidad y consistencia de las respuestas.

17 Conclusiones

El presente trabajo abordó el diseño e implementación de un sistema de recuperación de información para literatura científica que integra recuperación léxica mediante BM25F, recuperación semántica basada en embeddings densos, expansión de consultas, fusión de rankings mediante RRF y generación de respuestas asistida por modelos de lenguaje bajo un esquema RAG. Esta combinación proporciona un enfoque híbrido para la recuperación y exploración de información especializada.

Uno de los principales aportes del sistema es la integración de un flujo completo que abarca desde la adquisición dinámica de documentos hasta la presentación de resultados en una interfaz interactiva. La incorporación de búsqueda web automática, recomendación basada en historial y expansión de consultas mejora la cobertura y relevancia de los resultados en un dominio caracterizado por su complejidad semántica y constante evolución.

En conjunto, los resultados obtenidos muestran que la integración de técnicas clásicas de recuperación de información con modelos modernos de aprendizaje profundo y generación de lenguaje constituye una estrategia efectiva para mejorar el acceso al conocimiento científico. No obstante, persisten limitaciones relacionadas con la calidad del corpus, la evaluación de relevancia y la fidelidad de las respuestas generadas, que representan oportunidades para futuros trabajos.

References

1. Robertson, S., Jones, K.: Relevance weighting of search terms. *Journal of the American Society for Information Science*, 1976.
2. Robertson, S., Zaragoza, H., Taylor, M.: Simple BM25 Extension to Multiple Weighted Fields. In: *Proceedings of the Thirteenth ACM International Conference on Information and Knowledge Management (CIKM)*, 2004.
3. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
4. Cormack, G. V., Clarke, C. L. A., Buettcher, S.: Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *Proceedings of SIGIR*, 2009.
5. Lavrenko, V., Croft, W. B.: *Relevance Based Language Models*. SIGIR, 2001.
6. Lewis, P. et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020.
7. Ricci, F., Rokach, L., Shapira, B.: *Recommender Systems Handbook*. Springer, 2015.
8. Streamlit Inc.: *Streamlit Documentation*. <https://docs.streamlit.io>
9. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.
10. Cloudflare: *Workers AI Documentation*. <https://developers.cloudflare.com/workers-ai>